

System Blueprint (*a.k.a.* “Design Doc”)

TNPG: Restaurant Simulator

project: grocery simulator

Target ship date: {2026-06-01}

roster:

Name	Email	Primary Role	Secondary Role
Yu Lu	yul29@nycstudents.net	Front-end	Back-end
Ricky Lin	rickyl49@nycstudents.net	Middleware	Front-end
Jun Jie Li	junjiel26@nycstudents.net	Front-end	Art
Emily Mai	emilym384@nycstudents.net	Database Manager	Middleware

Summary

We plan to create a grocery store management simulator where the player will play in two roles: the cashier and the manager. Customers will come into the store daily and buy goods before going to the register to pay, the cashier will calculate and total the goods and give them back any leftover change. When the day ends, the manager will be responsible for buying new stock for tomorrow and order new upgrades for the store that will improve customer mood and store efficiency.

Problem Being Solved

Improves mental-math capabilities Improves decision-making Improves planning for the future!

Target Users

Who will use this system?

- Gamers interested in the simulator genres
- People interested in developing mental math skills

Why This Project Matters

We aim to improve the decision making and mental math skill of users through a fun and interactive format!!!

Minimum Viable Product (MVP) Scope

Core Features (Required for Final Submission)

Features that **must** be completed: 1. Day - Cashier system 1. Customer generation 1. Grocery stock system

Stretch Features (Only if MVP is Complete)

1. Night - Manager system
2. Dialogue and Unique NPCS
3. Music and cutscenes

Explicit Non-Goals

Features intentionally excluded: - explicit save system

Technology Stack

Layer	Selected Tool
Backend Framework	Flask
Frontend Framework	tailwind
Database	SQLite
Authentication	Flask sessions
ORM / DB Library	SQLAlchemy

Why This Stack Was Chosen

We considered our respective skillsets and the frameworks that we were comfortable with

Team Ownership Plan

Each member must own meaningful deliverables.

Team Member	Primary Ownership	Secondary Ownership	Specific Deliverables
Yu Lu	Templates, JS files	Flask routes	Core game loop and front-end
Jun Jie Li	Art assets	Templates, JS files	Art visuals
Ricky Lin	Flask routes	DB files, SQLITE tables	Flask routing between pages
Emily Mai	DB files, SQLITE tables	Flask Routes	Established database

Component map

To be added

Site map

To be added ““

Key User Stories

eg0

As a player, I want to be as rich as possible so that I can beat the objective of the game

eg1

As a developer, I want to make the game interactive and functional so that we can fulfill the pleasures of our users

eg2

As a tester, I want to encounter and play through different games so that i can encounter scenarios with unintuitive/unexpected behaviors that developers might want to address

Database Design

EMILY !

Testing Plan

ALWAYS check developer console for errors and consoleLogs opening during testing 1. Play through 1 Day, serve customers and use the register and gun to see if they work as expected 2. Play through 1 night, buying upgrades and stock, and see if it is updated the next day 3. return to the website in another instance (or forced f5) to see if saves are working

Timeline

Week 1 Goals: Design document, preliminary setup of repo, core game loop, save system

Week 2 Goals: Basic art assets added, mainscreen done, user save saved to database and read/setup during refreshes

Week 3 Goals: Stretch goals

Internal Deadlines:

Special customers, Add our products

Completion Criteria (*a.k.a.* “Definition of ‘Done’ ”)

Project is considered complete when all of the following are true: 1. Core game loop 1. Management system 1. Save system

Open Questions

Should music/audio be added?

Appendix

N/A # Other N/A